

第 4 集 · 约 18 分钟

服务端：async/await 与 axum

一个程序，怎么同时招呼好几台设备

配套阅读：书的第 4 章



这一集你会学到

1. async/await 和 tokio：等网络的时候把 CPU 让出来
2. axum 的路由、中间件和提取器：要什么就在参数里写什么
3. Arc<Mutex<T>>：好几个请求怎么安全地共用一张会话表
4. 防重放为什么要“锁两次”
5. 一个真实的坑：缺字段时 axum 返回的是 422，不是 400

一个程序，好几台设备

**手机 A 在上传视频，电脑窗口每 2 秒轮询一次，
手机 B 正在输口令。要给每台设备开一个线程吗？**

不用。线程大部分时间都在等网络。async 函数在等待时把 CPU 让出来，少量线程就能同时服务成千上万个连接。

每个请求都要过四关



谁来推 Future



发生了什么

1. 运行时 → Future: poll(); Future → 系统: 数据到了吗? 没有; Future 登记 Waker (叫醒器), 返回 **Poll::Pending**
2. 运行时不傻等, 去跑别的任务
3. 系统 → Waker: wake(); 任务回到运行时的待办队列
4. 运行时 → Future: poll(); 这次有数据, 返回 **Poll::Ready**

阻塞的活儿挪出去

crates/lanshare/src/server/rpc.rs

```
1  async fn list(st: &Arc<AppState>) -> Vec<u8> {
2      let st2 = st.clone();
3      let files = tokio::task::spawn_blocking(move || st2.storage.list()).await.unwrap_or_default();
4      let mut ids = st.files.lock().expect("锁不会中毒");
5      let files: Vec<Value> = files
6          .into_iter()
7          .map(|f| json!({ "id": ids.id_for(&f.name), "name": f.name, "size": f.size, "mtime": f.mtime })))
8          .collect();
9      reply(json!({ "ok": true, "files": files })))
10 }
```

手写分支 vs 路由表

Python v1 · server.py

```
1 def _get(self, url):
2     if url.path == "/":
3         return self._index(url.query)
4     self._require_login()           # 不通过就 raise
5     if url.path == "/api/info":
6         return self._send_json(200, {...})
7     ...
8     raise HttpError(404, "not found")
```

Rust v2 · server/mod.rs

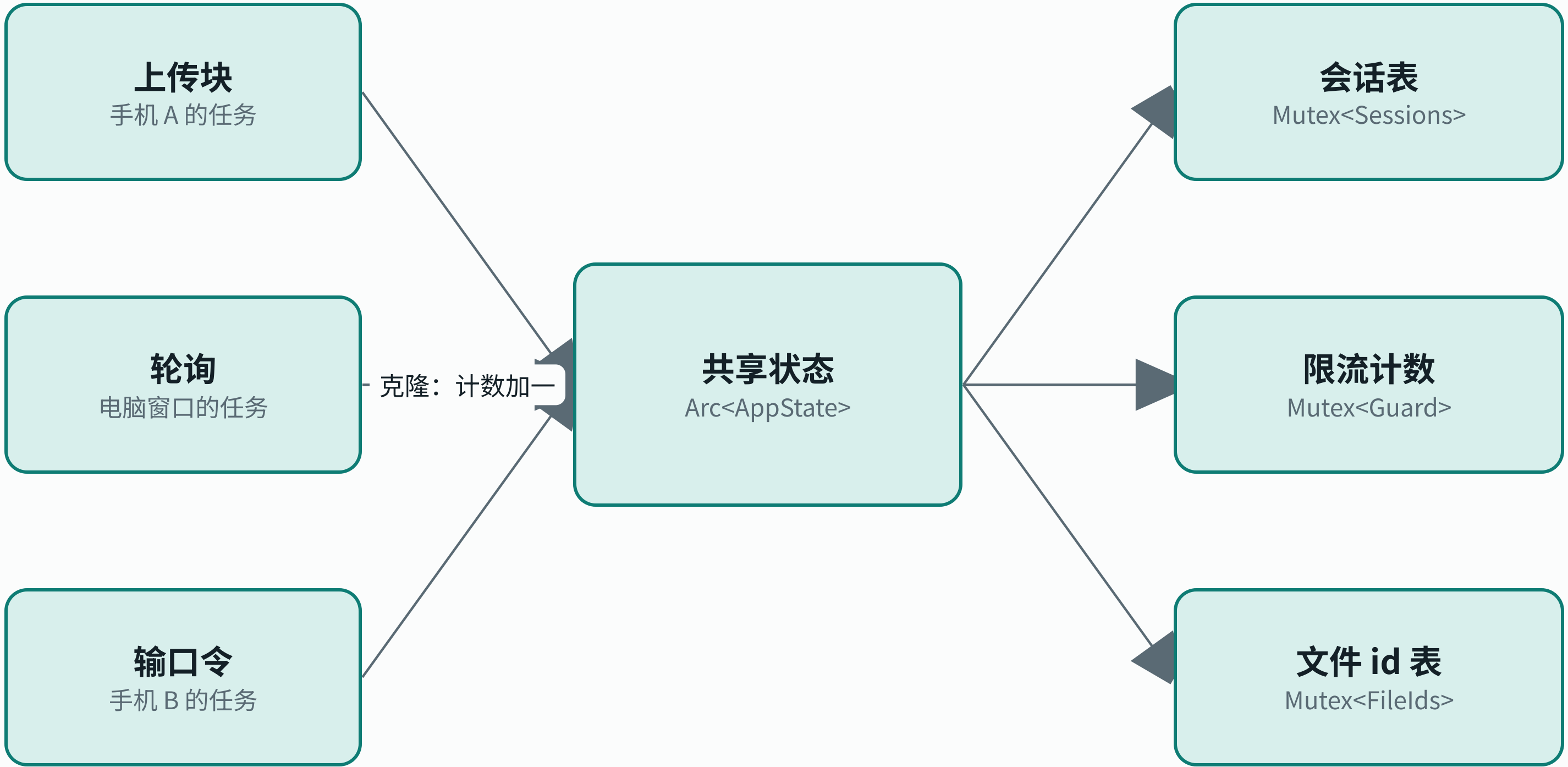
```
1 Router::new()
2     .route("/", get(assets::index))
3     // ...
4     .route("/api/hello", post(auth::hello).layer(DefaultBodyLimit::max(1024)))
5     // ...
6     .route("/api/rpc", post(channel::sealed))
7     .route("/api/up/{upload}/{index}", post(channel::sealed))
8     .route("/api/down/{file}/{index}", post(channel::sealed))
9     .layer(middleware::from_fn(check_origin))
10    .layer(middleware::from_fn(security_headers))
11    .layer(middleware::from_fn(check_host))
12    .with_state(state)
```

提取器：要什么写什么

crates/lanshare/src/server/auth.rs

```
1 pub(crate) async fn hello(  
2     state: St,  
3     ConnectInfo(peer): ConnectInfo<SocketAddr>,  
4     headers: HeaderMap,  
5     body: Bytes,  
6 ) -> Response {  
7     if !is_json(&headers) {  
8         return error(StatusCode::UNSUPPORTED_MEDIA_TYPE, "json_only");  
9     }  
10    let st = state.0;  
11    let ip = peer.ip();  
12    // ...  
13    let Some(body) = parse::<HelloBody>(&body) else { return error(StatusCode::BAD_REQUEST, "bad_request") };
```

一份状态，大家共用

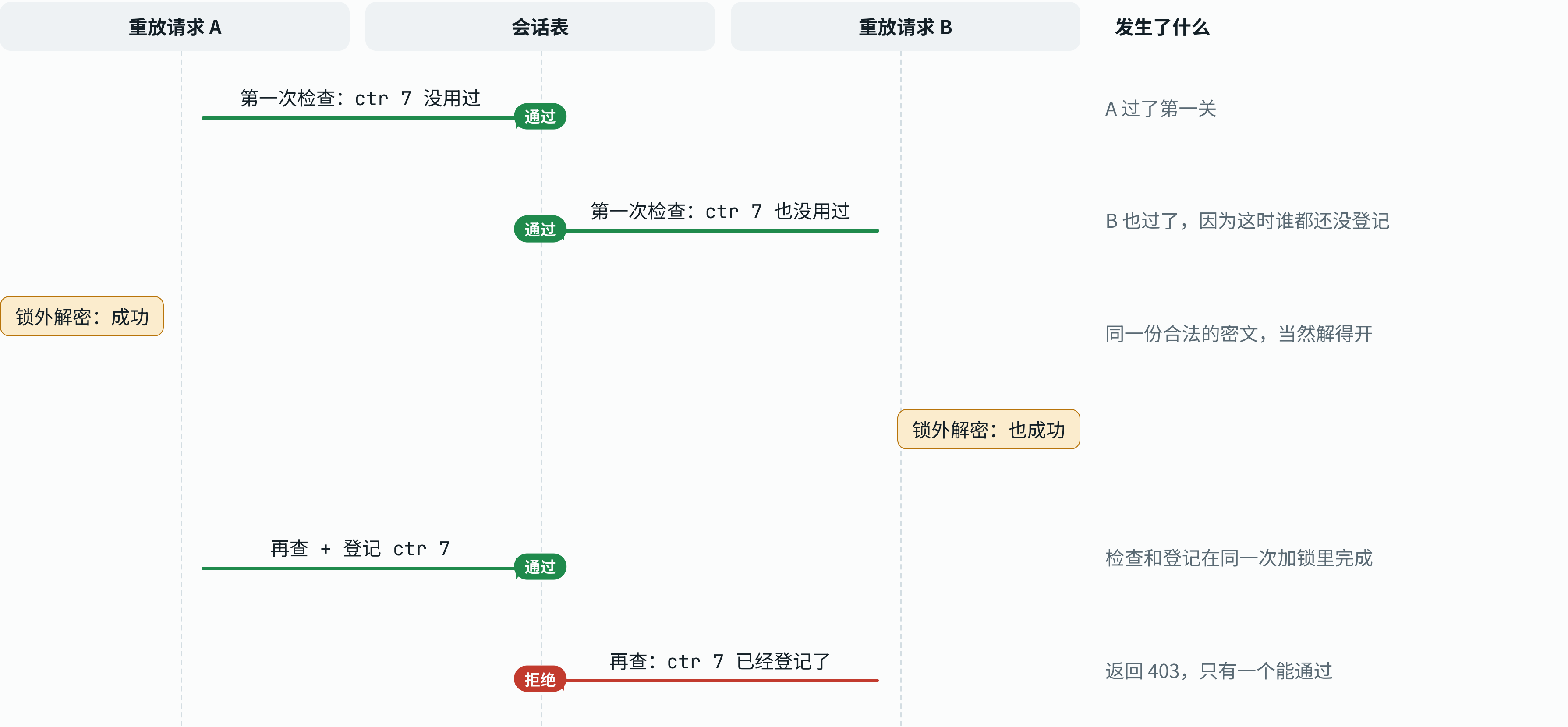


防重放：为什么锁两次

```
crates/lanshare/src/server/channel.rs
```

```
1 // 第一次加锁：取钥匙、预检计数器（不登记）。解密在锁外做，不拖慢其他设备。
2 let mut sessions = st.sessions.lock().expect("锁不会中毒");
3 let Some(session) = sessions.get(&cid) else {
4     return Err(StatusCode::UNAUTHORIZED);
5 };
6 if !session.window.check(ctr) {
7     return Err(StatusCode::FORBIDDEN);
8 }
9 Ok((cid, ctr, session.keys.clone()))
10 // ...
11 let plain = proto::open(&keys.c2s, ctr, &proto::req_aad(path), body).map_err(|_| StatusCode::FORBIDDEN)?;
12 // 第二次加锁：再查一次再登记，两个相同的重放请求并发到达时只有一个能通过。
13 let mut sessions = st.sessions.lock().expect("锁不会中毒");
14 let Some(session) = sessions.get(&cid) else {
15     return Err(StatusCode::UNAUTHORIZED);
16 };
17 if !session.window.check(ctr) {
18     return Err(StatusCode::FORBIDDEN);
19 }
20 session.window.commit(ctr);
```

两个一模一样的重放



动手：防重放窗口

计数器 1 ~ 32

已接受

窗口内

刚被拒绝

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

还没收到任何请求

发下一个

重发上一个（重放）

发一个很旧的

跳着发（+4）

重来

服务端的判断

踩坑：缺字段返回 422

crates/lanshare/src/server/auth.rs

```
1  /// 请求体解析成 JSON；格式不对、字段缺失一律当作 400，不暴露解析细节。
2  fn parse<T: DeserializeOwned>(body: &Bytes) -> Option<T> {
3      serde_json::from_slice(body).ok()
4  }
5  // ...
6  #[derive(Deserialize)]
7  pub(crate) struct HelloBody {
8      cid: String,
9      tag: String,
10 }
```

选择题

列共享文件夹时，为什么要把 `storage.list()` 放进 `spawn_blocking`？

A async 函数里不允许调用普通的同步函数

B 为了读得更快，多开几个线程一起读目录

C 读目录会阻塞，直接调用会占住 tokio 的工作线程

D 放进去以后就不用等它的结果了

选择题

列共享文件夹时，为什么要把 `storage.list()` 放进 `spawn_blocking`？

A `async` 函数里不允许调用普通的同步函数

允许，编译也能过。问题不在语法，而在它会卡住线程，别的请求跟着等。

B 为了读得更快，多开几个线程一起读目录

跟速度无关。目录还是一个线程在读，目的是别占住 `async` 的工作线程。

C 读目录会阻塞，直接调用会占住 `tokio` 的工作线程

对。同一个工作线程上的其他请求都得等。`spawn_blocking` 把它交给专门跑阻塞任务的线程池。

D 放进去以后就不用等它的结果了

还是要等，代码里写的就是 `spawn_blocking(...).await`。区别是等的时候，工作线程可以去服务别的连接。

选择题

防重放如果改成“加锁检查 → 锁外解密 → 加锁直接登记（不再检查）”，会出什么问题？

- A 没问题，第一次检查已经把重放拦住了
- B 两个一模一样的重放请求同时到达，都会被处理
- C 没问题，重放的密文本来就解不开
- D 会死锁

选择题

防重放如果改成“加锁检查 → 锁外解密 → 加锁直接登记（不再检查）”，会出什么问题？

A 没问题，第一次检查已经把重放拦住了

拦得住“先后到达”的重放，拦不住“同时到达”的：两个请求做第一次检查时，谁都还没登记。

B 两个一模一样的重放请求同时到达，都会被处理

对。两个都能通过第一次检查，又都能解密成功。第二次不再检查，就会各登记一次、各处理一次。

C 没问题，重放的密文本来就解不开

重放的是一份合法的密文，钥匙、计数器、路径都对得上，当然解得开。

D 会死锁

两次加锁之间锁已经放开了，不会死锁。真正的问题是并发下被处理两次。

想一想

为什么拿着 `std::sync::Mutex` 的锁时不能 `.await`？我们的代码是怎么避免的？

想一想

为什么拿着 `std::sync::Mutex` 的锁时不能 `.await`？我们的代码是怎么避免的？

`.await` 会让出线程，可锁还没放开。别的任务再来拿这把锁就会卡住，严重时整个运行时都会卡死。所以代码里的加锁都放在很小的作用域里：`precheck` 一返回，或者一个小小的 `{}` 块一结束，锁就自动释放，之后才去 `.await`。这叫 RAI，和 Python 的 `with` 块结束自动解锁是一回事。

这一集我们学了

- 🕒 async fn 返回一个 Future，tokio 负责 poll 它；阻塞的活儿交给 spawn_blocking
- 🕒 axum 三件套：路由表、中间件、提取器，要什么就在参数里写什么
- 🕒 Arc 管“大家共同拥有”，Mutex 管“同一时刻只有一个人改”；忘了加锁编译不过
- 🕒 防重放：先检查 → 锁外解密 → 再检查并登记，并发重放也只有一个能过
- 🕒 框架的默认行为要用测试锁住，比如缺字段返回 422

下一集

下一集：分块传输和抓包测试，1.5 GB 的视频怎么传，“加密了”又怎么证明